

Insertion in Middle of Linked List

Insertion in middle means:

- Insert node at a specific position
 - Insert after a specific node
-

Intuition

Suppose linked list is:

10 → 20 → 40 → 50

Insert 30 at position 3.

Result:

10 → 20 → 30 → 40 → 50

Main Idea

To insert in middle:

We must:

1. Reach node before insertion position
 2. Connect new node carefully
-

Visualization

Initial:

10 → 20 → 40 → 50

Create new node:

30 → NULL

Now carefully change links.

Step-by-Step Linking

Suppose temp is at 20.

Current:

20 → 40

Step 1:

newNode.next = temp.next

Becomes:

30 → 40

Step 2:

temp.next = newNode

Final:

20 → 30 → 40

Entire list:

10 → 20 → 30 → 40 → 50

VERY IMPORTANT ORDER

Correct:

```
newNode.next = temp.next;  
temp.next = newNode;
```

Wrong:

```
temp.next = newNode;  
newNode.next = temp.next;
```

Why wrong?

Because:

- temp.next already changed
 - original next node lost
 - creates wrong connection/self-loop
-

Algorithm

Suppose insertion at position pos.

Steps:

1. Create new node
 2. Traverse till (pos - 1)th node
 3. newNode.next = temp.next
 4. temp.next = newNode
-

Position Understanding

Position Meaning

- 1 Beginning
 - 2 After first node
 - 3 After second node
-

Example

Insert 25 at position 3.

List:

10 → 20 → 30 → 40

Traversal:

- Reach node 20

Now:

newNode.next = 30

20.next = 25

Final:

10 → 20 → 25 → 30 → 40

Time Complexity

Operation	Complexity
-----------	------------

Insert in Middle	$O(N)$
------------------	--------

Because traversal needed.

Python Code

```
class Node:
```

```
    def __init__(self, data):  
        self.data = data  
        self.next = None
```

```
class LinkedList:
```

```
    def __init__(self):  
        self.head = None
```

```
  
    # Insert at end
```

```

def insert_end(self, data):
    newNode = Node(data)

    if self.head is None:
        self.head = newNode
        return

    temp = self.head

    while temp.next:
        temp = temp.next

    temp.next = newNode

# Insert at position
def insert_middle(self, pos, data):
    newNode = Node(data)

    # Insert at beginning
    if pos == 1:
        newNode.next = self.head
        self.head = newNode
        return

    temp = self.head

    # Reach (pos-1)th node
    for i in range(pos - 2):
        temp = temp.next

    newNode.next = temp.next
    temp.next = newNode

def display(self):
    temp = self.head

    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next

    print("NULL")

```

```

ll = LinkedList()

```

```

ll.insert_end(10)
ll.insert_end(20)
ll.insert_end(40)

```

ll.insert_end(50)

ll.insert_middle(3, 30)

ll.display()

C++ Code

```
#include <iostream>
using namespace std;
```

```
class Node {
public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
};
```

```
class LinkedList {
public:
    Node* head;

    LinkedList() {
        head = NULL;
    }
```

```
void insertEnd(int data) {
    Node* newNode = new Node(data);

    if(head == NULL) {
        head = newNode;
        return;
    }

    Node* temp = head;

    while(temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
}
```

```

// Insert at middle
void insertMiddle(int pos, int data) {
    Node* newNode = new Node(data);

    // Insert at beginning
    if(pos == 1) {
        newNode->next = head;
        head = newNode;
        return;
    }

    Node* temp = head;

    // Reach (pos-1)th node
    for(int i = 0; i < pos - 2; i++) {
        temp = temp->next;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}

void display() {
    Node* temp = head;

    while(temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL" << endl;
}

};

int main() {
    LinkedList ll;

    ll.insertEnd(10);
    ll.insertEnd(20);
    ll.insertEnd(40);
    ll.insertEnd(50);

    ll.insertMiddle(3, 30);

    ll.display();

    return 0;
}

```

Java Code

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

class LinkedList {
    Node head;

    void insertEnd(int data) {
        Node newNode = new Node(data);

        if(head == null) {
            head = newNode;
            return;
        }

        Node temp = head;

        while(temp.next != null) {
            temp = temp.next;
        }

        temp.next = newNode;
    }

    // Insert at middle
    void insertMiddle(int pos, int data) {
        Node newNode = new Node(data);

        // Insert at beginning
        if(pos == 1) {
            newNode.next = head;
            head = newNode;
            return;
        }

        Node temp = head;

        // Reach (pos-1)th node
        for(int i = 0; i < pos - 2; i++) {
```

```

        temp = temp.next;
    }

    newNode.next = temp.next;
    temp.next = newNode;
}

void display() {
    Node temp = head;

    while(temp != null) {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
    }

    System.out.println("NULL");
}

public static void main(String[] args) {
    LinkedList ll = new LinkedList();

    ll.insertEnd(10);
    ll.insertEnd(20);
    ll.insertEnd(40);
    ll.insertEnd(50);

    ll.insertMiddle(3, 30);

    ll.display();
}
}

```

Dry Run

Insert 30 at position 3.

Initial:

10 → 20 → 40 → 50

Step 1:

Reach node before insertion.

temp = 20

Step 2:

newNode.next = temp.next

Means:

30 → 40

Step 3:

temp.next = newNode

Final:

10 → 20 → 30 → 40 → 50

Interview Explanation

Why insertion in middle is $O(N)$?

Because:

- We must traverse to required position
 - Traversal takes linear time
-

Common Mistakes

Mistake 1: Wrong order

Wrong:

```
temp.next = newNode;  
newNode.next = temp.next;
```

Creates wrong connection.

Mistake 2: Forgetting position 1

If inserting at position 1:

- It becomes insertion at beginning

Need special handling.

Edge Cases

1. Empty Linked List

head = NULL

Only position 1 valid.

2. Insert at Beginning

pos = 1

Handle separately.

3. Insert at Last

10 → 20 → 30

Insert 40 at position 4.

Result:

10 → 20 → 30 → 40

Difference Between Insertions

Type	Traversal Needed	Complexity
Beginning	No	O(1)
Middle	Yes	O(N)
End	Yes	O(N)

Important Interview Point

Insertion in linked list is mainly about:

Careful pointer manipulation

The most important line:

```
newNode.next = temp.next;
```

This preserves remaining linked list.

Without this:

- Remaining nodes get disconnected.
-

Real World Usage

Insert in Middle Used In:

- Music playlist insertion
- Text editor cursor insertion
- Browser tabs arrangement
- Task scheduling systems

